

GraphOne Intelligence Pipeline Architecture

Technical Deep-Dive: Scalability, Rate-Limiting, Freshness Tracking, and Data Strategy

1. Scale Strategy: Scaling to 500,000+ Records

Current Concurrency Architecture: The prototype implements non-blocking asynchronous concurrency using Python's `asyncio` and `aiohttp`. Concurrency is managed via explicit semaphores (e.g., `asyncio.Semaphore(10)` for arXiv API paper extraction in `src/scrapper/papers.py`) and HTTP connection pooling. Startups and products are scraped concurrently using `asyncio.gather`, while Y Combinator Algolia directory fetching utilizes multi-page pagination (1,000 hits/page) to extract 999 verified companies rapidly.

Scaling to 500,000+ Records (Distributed Architecture): To scale from ~3,000 to 500,000+ records across millions of entities, the single-process `asyncio` loop generalizes into a horizontally scalable, distributed queue system:

- **Distributed Queue & Worker Pools:** Celery / ARQ workers backed by Redis or AWS SQS. Domain-specific queues (`queue:papers`, `queue:startups`, `queue:products`, `queue:freshness`) allow independent scaling of crawler nodes.
- **Distributed Rate Limit Governor:** Centralized Token Bucket / Leaky Bucket rate limiter implemented in Redis (via `redis-py` / Lua scripts) ensuring IP and API compliance across 100+ concurrent crawler containers.
- **Incremental Partitioned Persistence:** Worker nodes flush records incrementally in 100-record chunks to JSONL staging logs, followed by `async bulk COPY / UPSERT` into PostgreSQL.

2. Handling 413 Payload Errors & 429 Rate Limits

Chunking & Token Limits (Avoiding 413 Payload Too Large): Large document abstracts and raw web text are managed in `src/llm/orchestrator.py` using `tiktoken` token counting (`cl100k_base` encoding). A strict `TOKEN_LIMIT = 6000` threshold with a `MAX_TOKEN_CAP = 8000` ceiling triggers paragraph-boundary text splitting (`text.split('\n\n')`) before reaching LLM API endpoints, completely eliminating 413 Payload Too Large failures.

429 Rate Limit Mitigation & Real Production Errors Resolved:

- **Papers with Code API 429s:** During arXiv paper collection, Papers with Code API enforced strict HTTP 429 rate limits. We implemented stateful short-circuiting (`PWC_RATE_LIMITED = True`) upon receiving HTTP 429/403, immediately bypassing PWC for remaining paper entries to prevent pipeline stalls.
- **Direct HTML Fallback Extraction:** Primary GitHub repository discovery reads arXiv abstract page HTML directly via regular expressions (`re.search(r'https?:/github.com/...')`), bypassing external APIs entirely and achieving zero rate-limit overhead.
- **GitHub REST API 403 Rate Limits:** When GitHub REST API rate limits were encountered during stargazer fetching, the system fell back to parsing raw GitHub repository HTML (`id='repo-stars-counter-star'`) using custom user-agent headers, successfully extracting star counts without API authentication.

3. Freshness Tracking & Distributed Deduplication

24-Hour Filtering & Assertion Mechanism: In `src/scrapper/freshness.py`, news items and job listings are evaluated against a strict rolling 24-hour cutoff (`cutoff_time = now_utc - timedelta(hours=24)`). Publication timestamps are parsed using `dateparser` and converted strictly to UTC timezone-aware objects. A hard runtime assertion check validates every record before output, raising a `ValueError` if any stale entity older than 24 hours escapes.

Distributed Deduplication at Scale: In a multi-node distributed crawler, duplicate detection across nodes is handled via:

- **Redis Key Hashing (SETNX):** Global URL deduplication using Redis `SETNX` on `seen:url:{sha256(source_url)}` with a 48-hour TTL matching the freshness window.
- **RedisBloom Filter:** Probabilistic Bloom filters in Redis provide $O(1)$ time complexity duplicate checks for 1B+ records with sub-millisecond latency and minimal memory footprint.

4. Storage Strategy & Graph/Vector Relationship Mapping

PostgreSQL Core Relational Layer: Postgres serves as the primary data store for structured entities (`'Startups'`, `'Products'`, `'ResearchPapers'`, `'Jobs'`, `'News'`), providing ACID compliance, schema enforcement via `Pydantic v2`, and transactional integrity. Composite unique indices on (`source_name`, `source_url`) guarantee idempotent upserts.

Proposed Graph & Vector Architecture for Relationship Mapping:

- **Vector Layer (pgvector / Qdrant):** Store 1536-dimensional embeddings for research paper abstracts, startup descriptions, and product capabilities. Enables semantic similarity search, automated tagging, and Retrieval-Augmented Generation (RAG) queries.
- **Graph Layer (Neo4j / AWS Neptune):** Map multi-hop entity relationships across datasets: (Startup)-[:BUILT]->(Product), (Paper)-[:AUTHORED_BY]->(Author), (Job)-[:POSTED_BY]->(Startup). Enables multi-hop graph queries such as *'Identify YC startups with AI products whose authors published highly-cited arXiv papers in the last 24h.'*

5. Final Pipeline Performance Metrics

Metric / Tab Name	Count / Value	Verification & Architecture Notes
Research Papers	1,000	100% real arXiv papers; 164 with non-null GitHub stars
Startups	999	100% verified YC AI directory startups (Algolia API index)
Products	1,000	100% distinct public launches (Show HN / Product Hunt)
Jobs (24h)	9	100% strictly within 24h freshness window (UTC verified)
News (24h)	25	100% strictly within 24h freshness window (UTC verified)
Entity Mapping Log	2,999	Breakdown: new: 1813, llm: 1125, exact: 57, fuzzy: 4
Automated Test Suite	6 / 6 Passed	100% pass rate (`pytest tests/ -v` in 1.41s)
LLM Tier Orchestration	1,125 Calls	Tier 1 (Gemini): 1,125 live calls; Tier 2 (Groq): Ready; Tier 3: Unconfigured
Google Sheets Export	Verified Live	All 6 worksheet tabs exported via gspread batch_update